Advanced Machine Learning for Physics student project: RECENTRE

Francesco Passante and Eugenio Nicotina
(Dated: July 14, 2026)

We build a machine learning framework to train, validate, finetune models and perform data analysis for the RECENTRE (REal-time motion CorrEction in magNeTic REsonance) project. We first reproduce the original results of the RECENTRE project based on a GRU network through a config-oriented software design, then we introduce and evaluate more modern machine learning architectures, studying in what regimes they are most effective and whether they're better candidates for real time deployment in MRI machines. Model size, inference latency, robustness to noise and uncertainty calibration are among the many different metrics by which we evaluate the performances of each of the proposed architectures.

I. INTRODUCTION

The RECENTRE project is concerned with real time prediction and correction of head motion parameters. A crucial fact to take into account when building models for this task is the inference latency of the model. The temporal distance between the frame recording and the subsequent next frame prediction must be of the order of 100 ms, so the inference latency must be well below this limit to account for other software- and hardware-related latencies.

The main goals of the present project are to reproduce the original paper's ([1]) results, which are based on a GRU network, and evaluate modern machine learning architectures such as transformers, TCNs, and physics-informed models to study whether they are better candidates for the project's task.

Given the large amount of models and the large amount of training runs (different model hyperparameters, different train / test tasks, data augmentation, etc...) we introduced a config-based software design pattern to keep the code and the output data well organized. This way, new models can be implemented and registered in a model registry dictionary; the hyperparameters of the model, the preprocessing pipeline of the data and the training details (epochs, learning rate, ...) can all be specified in a minimal, easy-to-read configuration file. This way, we built a solid end-to-end pipeline from the original raw data and model specification to the full training and evaluation loop.

II. DATASET

The raw dataset is taken from the Human Connectome Project (HCP), which consists in navigator data taken by MRI machines with a sampling frequency of 720 ms. Each 'frame' has 6 motion parameters: 3 of them are translation parameters describing the head position of the patient, the remaining 3 are rotation parameters indicating the inclination of the patient's head. The dataset contains three different scenarios: the "R" (resting) one, where patients were resting while the MRI scan was running, then the "M" (memory) one, where patients were recollecting memories, and the "L" (language) one, where patients were actively talking. There are a total of 3221 time series among the three tasks. Each time series is associated with a patient. The preprocessing pipeline discards 140 time series, so that 1027 patients are all associated with the 3 time series of the "R", "M", "L" tasks. The three tasks have different time lengths. The R data is 1200 frames long, while the M and L are 405 and 316 frames long, respectively. Lots of effort was put in the thesis and paper to study different data splits for training and testing, e.g. considering the "cross-task" scenario or "cross-patient" scenario. We build on their findings (no substantial difference in performance among the different scenarios) by:

1. Developing a minimal, simple pipeline to choose training and testing tasks and choosing whether to use different splits of patients
2. Standardizing all results to the same scenario: train on all tasks (R+M+L) and test on all tasks (R+M+L), while keeping disjoint sets of patients to avoid any possible leakage

The thesis' showed that all cross-task combinations or cross-patients combinations had essentially identical performances. This suggested to us that training a single model with all three tasks would probably yield better performances (more data, covering the whole training distribution) and facilitate inference, as we have a single model regardless of the testing task. Our analysis indeed showed that this was the case.

A subsequent analysis on physics-informed models has shown that exploiting additional kinematic data such as velocity and acceleration parameters substantially improved test performances. So, 6 additional columns containing velocity parameters and 6 columns containing acceleration parameters for the 6 different dimensions are kept and used.

A. Data augmentation

The dataset contains roughly 2 million input window - frame target pairs, while our models have $O(100k)$ parameters. Data augmentation is therefore not a priority

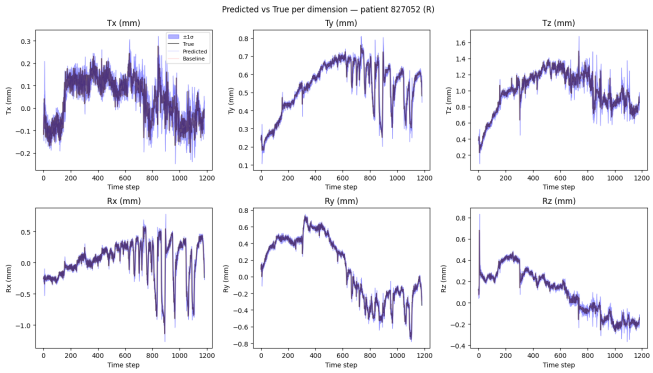


FIG. 1: Example patient time series showing predicted vs. true head-motion parameters across the six degrees of freedom.

for the project. Nevertheless, we investigated whether data augmentation could provide a benefit to model performances. Since the translation and rotation parameters are physically linked, there is no much room for independent transformations of the input data. However, two possible transformations are considered for data augmentation: time reversal and sign flip. These two augmentation techniques can quadruple the initial training dataset.

B. Preprocessing

The preprocessing pipeline consists in extracting the raw data from the HCP files, converting degrees to radians, ensuring a standardized time length for each of the different tasks, and ensuring that each patient is associated with its three runs. The preprocessing pipeline is meant to be run only once, as it builds three processed dictionaries (one for each task) that associate each patient id with its time series for that task. All successive scripts read from the processed datasets.

III. METHODOLOGY

A. Pipeline

All the tested models follow the same protocol:

1. Model definition and registration
2. Training config compilation
3. Training loop
4. Evaluation loop and data analysis
5. Optional fine-tuning

After model definition, typically as a torch module, a training config is necessary to run the training loop. The model hyperparameters, training and testing tasks, split percentages, data augmentation, loss function, training epochs, learning rate and so on are all specified in the training config, so that all the relevant parameters are easy to read, especially for systmeatic comparisons between many different models and runs.

After the training is done, the model weights, the optimizer and scheduler states, the training config, the training and testing tasks and patient ids are stored in a checkpoint to be used for testing, finetuning or resuming training.

B. Metrics

The fundamental metric that describes the prediction error of a single frame is called frame displacement (FD). It is defined as

$$FD(t) = \sum_{i=1}^{3} |y_i(t) - \hat{y}_i(t)| + 50\text{mm} \sum_{i=4}^{6} |y_i(t) - \hat{y}_i(t)| \quad (1)$$

it is essentially an absolute error that combines translation error ($i = 1, 2, 3$) and rotation error ($i = 4, 5, 6$), where the radians are scaled by 50mm, the average skull radius. This metric is a single scalar describing the absolute error of the prediction relative to the true position. Since the sampling rate is 720ms, the lag1-model that predicts the last observed position as the next one is already a solid baseline. We'll refer to the lag-1 model as the baseline model. Naturally, we can define the FD_{base} as the frame displacement for the lag1-model:

$$FD_{base}(t) = \sum_{i=1}^{3} |y_i(t) - y_i(t-1)| + 50\text{mm} \sum_{i=4}^{6} |y_i(t) - y_i(t-1)| \quad (2)$$

Another useful metric is the FD_{gain}, that is the relative difference between FD_{base} and FD_{model}:

$$FD_{gain} = \frac{FD_{base} - FD_{model}}{FD_{base}} \quad (3)$$

We aim for large, positive FD_{gain}.

Another useful metric is the percentage of frames that is improved (i.e. lower frame displacement) with respect to the baseline. Analogously, we can look at the percentage of patients with an average $FD_{gain} > 0$.

C. Loss function

We use the same form of the loss function as the original work:

$$L = L_{NLL} - \beta FD_{gain} \quad (4)$$

Where L_{NLL} is the gaussian negative log likelihood term that is needed for both prediction accuracy and uncertainty calibration.

Using FD_{gain} in the loss is a way to penalize the model if it does worse than the baseline model, especially when the baseline model is performing well in that particular frame. This loss term is essentially specifying that we'd rather accept large FDs in frames where the baseline has poor performance than have frames where the model is worse than the baseline.

We also reduce the learning rate on loss plateau and use early stopping.

D. Finetuning

Finetuning in this project is intended as a per-task, per-patient finetuning. After a model has been pre-trained on many different time series, it can be additionally fine-tuned on the first part of a time series so that the model can be adapted to the particularities of the patient and task under observation. The original work has found a substantial improvement in test performance after fine-tuning, relative to the pre-trained generalist model.

The finetuning implementation idea is the same as the original work one. It uses a modified loss function:

$$L = L_{MSE} - \beta FD_{gain} + \lambda L_{L2SP} \tag{5}$$

It differs from the pre-training loss function because it lacks an uncertainty calibration term (which is fine because the uncertainty head is frozen) and it has a L^2 starting point (L2SP) term that penalizes significant drifts from the pre-trained model:

$$L_{L2SP} = \sum_i (w_i - w_i^0)^2 \tag{6}$$

where w_i^0 is the i-th pretrained model weight.

Our own implementation aims to standardize this procedure across all the implemented models, and make all the finetuning parameters readily accessible in a dedicated finetuning config file. All models share the same output maps: two fully connected heads from the latent space of the model to the (mean, var) predictions. Using the proper name convention, all models are therefore finetuned in the same way, using different learning rates for the tunable head and the rigid body.

IV. REPRODUCTION OF PREVIOUS RESULTS

The first part of the project is concerned with reproducing the results of the previous thesis and paper. First, we refactored the codebase, fixed minor bugs and organized it in a config-oriented design, with standardization of train/test scenarios, model definitions, finetuning pipeline, model / data / train hyperparameters, data analysis and visualizations.

Then, we focused on reproducing the results from the previous work:

1. Paper main result: $R^2 > 0.87$ for all the 3 tasks and 6 dimensions.
2. Thesis result: roughly 70% frames have $FD_{gain} > 0$.
3. Thesis result: positive FD_{gain} frames percentage rises to 75% for the finetuned model.

We first reproduced the original results by considering the per-task training and testing, but we soon decided to directly improve these results by adopting some fairly obvious refinements on top of the original structure. First, we dropped the cross-task scenario debate and decided to train and test a single model on all three tasks. We used different patients for the train, validation and test splits to avoid any data leakage. The hyperparameters of the model are left the same. The results are visible in Fig. (2)

We also reproduced the per-patient fine-tuning result of the previous work. Fig. (3) summarizes the effect of fine-tuning, and Fig. (4) compares each patient before and after fine-tuning.

V. RESULTS

A. β scan

The previous work used a fixed (arbitrary) choice of $\beta = 0.1$. We investigated the performances of the original GRU model as β is swept over many orders of magnitude. In particular, we focused on the most relevant metrics that could sharply depend on the relative importance between the negative log likelihood and the FD_{gain} term. Since the negative log likelihood term is the only one explicitly constraining the uncertainty of the prediction,we expect that as β grows, the uncertainty calibration will get worse. On the other hand, FD_{gain} and the percentage of improved frames (relative to baseline) should rise with β. Our empirical results are summarized in Fig. (5).

As expected, uncertainty calibration degrades as β grows near and past $\beta = 1$, while FD_{gain} rises and then decreases again when FD_{gain} is over-weighted in the loss. This is probably an overfitting phenomenon specific to the FD_{gain} term: even though the loss function at very high β is essentially a maximization of FD_{gain}, this doesn't generalize well to the validation/test set. Across different scans, the three values $\beta = 0.1, 0.5, 1$ showed the best performances. We chose to use $\beta = 0.5$ as a middle ground between the optimally calibrated but lower FD_{gain} focused $\beta = 0.1$ and the less calibrated but higher FD_{gain} focused $\beta = 1$.

B. Features augmentation

For each frame HPC data provides the six positions of the head and the relative 1-strided differences (the

TABLE I: Effect of the jerk-penalty weight γ on test-set performance (GRU, sequence length 32).

γ	Mean FD	Mean FD_{gain}	% Frames > base
0	0.1268	0.1778	78.4
0.001	0.1267	0.1785	78.5
0.01	0.1267	0.1767	78.2
0.1	0.1280	0.1786	79.0
0.3	0.1273	0.1798	78.8
0.5	0.1285	0.1769	78.7
0.7	0.1312	0.1693	78.5
1	0.1363	0.1514	77.3

velocity). However since the model is fed with 2-strided positions, also velocity has to be explicitly recomputed:

$$v(t) = x(t) - x(t-2), \tag{7}$$

and not v(t) = x(t) - x(t-1) as the HPC dataset provides. Same story holds for acceleration, the 2-strided acceleration is defined as:

$$a(t) = v(t) - v(t-2) = x(t) - 2x(t-2) + x(t-4). \tag{8}$$

The effects of these features augmentation are showed in Table II: adding velocity dramatically improves performances, whereas adding also acceleration does not seem to add any information to the model. Nevertheless we kept both of them in the train and evaluation of all the models.

TABLE II: Effect of features augmentation on test-set performance.

Augmentation	FD_{pred}	FD_{gain}	% Frames > base
None	0.134	0.145	76.5
Vel	0.129	0.172	78.5
Vel+acc	0.129	0.171	78.1

C. Data augmentation

Next, we investigated whether data augmentation could be helpful for this project. Since the translation and rotation parameters are physically related, there is little room for independent transformation of the motion parameters. We investigated two physically plausible transformations: time reversal and negation (sign flip).

From the empirical data in Table III we find that, while time reversal augmentation degrades the performances on the (non augmented) test set, negation significantly improves them. From now on, we always train the models with negation-augmented data.

TABLE III: Effect of data augmentation on test-set performance.

Augmentation	FD_{pred}	FD_{gain}	% Frames > base
None	0.143	0.097	73.0
Time-reversal	0.146	0.078	70.9
Negation	0.141	0.114	75.2
Both	0.145	0.085	71.9

D. γ scan (jerk penalty)

Since the task of interest is a physical motion prediction task, we investigated whether enforcing a kinematic prior could help the model find a better solution. Indeed, injecting symmetry priors or inductive biases into ML models is widely known to improve performance and/or efficiency in many applications (that's what geometrical deep learning is all about), e.g. enforcing SE(3) equivariance in mesh classification or gauge equivariance in lattice gauge theory models. For tasks that do not have an exact symmetry to enforce, an alternative approach is to inject the physical bias in the loss function. In our case, we studied whether a jerk penalty term in the loss could help the model explore more physically plausible solutions and thus lead to better optimization and generalization. We chose to use a jerk penalty because our data is rapidly fluctuating, so using velocity- or acceleration-related penalties probably wouldn't help. Jerk penalty is a bit more theoretically grounded, as it captures the muscles' ability to produce smooth changes in applied force, while still penalizing very rapid changes in movement.

We added the term

$$L_{jerk} = \gamma J(t) \tag{9}$$

to the loss function, where the jerk $J(t)$ is computed via finite diffirences as:

$$J(t) = a(t) - a(t-2) = x(t) - 3x(t-2) + 3x(t-4) - x(t-6) \tag{10}$$

Where the 2-strided input is kept to avoid using in-between frames that the model never sees. We then swept γ over many orders of magnitude to study its effect on test performances for the GRU model. The results are shown in Table I.

We see from the results that we gain at most 0.002 ($\sim$ 1%) from the base model. This very small gain is not sufficient to conclude that the jerk penalty improves prediction accuracy. A possible future extension of this project could be to train many models and to associate a statistical error to the test performance so that an actual statistical edge could be confirmed or ruled out.

E. TCN

The first new model we implemented is a temporal convolutional network (TCN). This network uses temporal convolution kernels at various level of dilations to create causal masks (i.e. the kernel is 0 for frames that are in the future with respect to the output frame). The last frame is then taken and linearly projected by two separate heads to produce (mean, var). Like all the architectures we implemented, the model predicts a residual with respect to the last position. A schematic diagram of the network forward is found in Fig. (6).

F. Transformer

The next model we implemented is a basic encoder-only transformer with multi-head self attention across the time frames. Just like in the TCN, the last frame of the output of the transformer is fed to two linear layers that predict the shift of the next frame and the corresponding variance. We use fixed, sinusoidal positional embeddings. For all attention-based architectures, which have permutation invariance naturally baked-in, time series forecasting is a hard task. Positional embeddings are necessary but often not sufficient to give a clear and unambiguous time ordering to the time series fed to the transformer. Many transformer-based architectures aim to solve this problem with various ideas. One of them is the PatchTST architecture.

G. PatchTST

PatchTST [2] is the state-of-the-art transformer model for time series forecasting in many different benchmarks. There are two main differences from a basic transformer model:

- Tokens fed to the attention mechanism are not single-frame, but multi-frame objects, called "patches".
- The transformer is channel-independent.

The patches are summed with learnable positional embeddings and then treated as standard tokens in a multi-head self-attention mechanism. All the final patches are used for prediction, unlike the classical transformer where only the last frame is used.

H. NLinear

A 2022 paper by Zeng et al [3] showed that "embarassingly simple" linear models could beat transformer-based models by a big margin in many time series forecasting tasks. We implemented the NLinear model proposed in that paper. It is a very light model, with $\mathcal{O}(\mathrm{T})$ parameters. The architecture philosophy is to center the whole window with respect to the last frame and apply a simple channel-independent linear map between the whole sequence and the predicted frame. In our case, there are 6 linear layers for the mean head and 6 linear layers for the variance head.

I. DLinear

The other model proposed in the Zeng paper is the DLinear model. It is among the lightest models as well. The idea is to decompose the global movement (trend) with respect to the high frequency one (season) by shifting a convolutional kernel along the window to extract a moving average. The original length-T sequence is therefore decomposed as a length-T trend component and a length-T season component. Then 3 linear ($T \to 1$) layers are applied separately to the trend, season and the original data, the first two outputs are summed to predict the next frame mean, while the latter to predict the variance. It is still a channel-independent model.

J. TSMixer

Our analysis (see later) showed that channel-independent models such as PatchTST, Nlinear and Dlinear don't have good performances on our task. A slighty more complicated architecture, inspired by the Zeng paper, was proposed by the Google Cloud AI team in 2023: TSMixer. The Time-Series Mixer [4] is composed mainly of MLP layers but with the advantage of alternating time-mixing layers with channel-mixing layers. A schematic picture of the TSMixer architecture is shown in Fig. (7). A number of TSMixer layers are applied in sequence, each of them composed of a multi-layer perceptron among the frames and another among the features. This model should allow for richer channel-mixing operations that our task seems to require.

K. Conformer

A conformer model [5], originally introduced for speech recognition, is an architecture that combines transformers and convolutional neural networks. The former is specialized to analyze long range patterns, while the latter is specialized to analyze short range ones, making the hybrid architecture extremely powerful and indeed the most successful one in our study. It inherits the multi head self attention mechanism and the positional embeddings from the transformer while it inherits the weight sharing of the CNN.

Concretely, each conformer block is a sandwich: a feed-forward module, followed by the multi head self attention

module, then the convolution module, and finally a second feed-forward module. The self attention module is the classical self-attention of other transformer architectures. The convolution module uses a 1x1 convolution first to mix features at a fixed timestep, then a gate is applied. Then, a channel-independent convolution among time frames is applied.

The whole idea of the conformer is to use attention to learn long range interactions and convolution to learn short range, seasonal changes.

L. Mamba

State space models recently recieved a lot of interest, mainly due to their representation learning ability. SSMs have a clear physics-inspired architecture, where a hidden state is progressively updated based on the input sequence. Like RNNs, however, they historically suffer from the lack of ability to have content-aware reasoning, since the update parameters are not content-dependent, but fixed after training. Recent SSMs like Mamba address this exact problem by allowing a data-dependent selection mechanism, taking the best advantage of transformers (content-dependent reasoning) with the best advantage of RNNs (linear scaling with sequence length). We therefore tried to implement a Mamba architecture for our multivariate time series forecasting task. Very schematically, a Mamba model updates its hidden state via three content-dependent quantities Δ, B, C and a content-indpendent quantity A. In particular, the fundamental update equation is:

$$h_{t+1} = e^{\Delta(x_t)A} h_t + \Delta(x_t) B(x_t) f(x_t) \tag{11}$$

Since A is a negative matrix, large $\Delta(x_t)$ means the previous hidden state is forgotten, and the current input contributes a lot to the hidden state, while small $\Delta(x_t)$ means the previous hidden state is retained, and the current input doesn't contribute much to it. This content dependance of Δ on $x(t)$ makes the mamba content-dependent and thus achieves optimal results in many tasks where transformers are very good at, while having a linear scaling vs. the quadratic scaling of standard attention. Our Mamba implementation is indeed the second best model of our benchmark, despite our implementation not being fully optimized.

M. Model comparisons

We trained 9 different architectures with 4 sequence lengths each. We compare all of them among very different metrics, from test accuracy to model size to noise robustness. In particular, we rank the models based on FD_{gain}, but we report, for all of them: number of parameters, size, FLOPs, latency, throughput, FD, FD_{gain}, $\%FD_{gain} > 0$, NLL, $\Delta FD@0.3\sigma$ (relative increase in FD when 0.3 σ noise is added to the data), tolerance to noise (σ of the injected noise at which the model becomes worse than the clean baseline). We report the full results in Table V.

To have a more visual look at the data, we create a set of Pareto-optimal frontier visualizations. We plot accuracy (FD_{gain}) on the y-axis and cost (latency, FLOPs, size) on the x-axis. We also do a scatterplot of FD_{gain} vs. noise tolerance, although the clear linear term simply shows that all models lose accuracy in the same way when noise is added, there is no clear 'noise-robust' winner, just a better model overall which needs more noise to be brought down in accuracy. The plots are shown in Fig. (9).

As we can see, the conformer model achieves the best test accuracy out of all the models. Second is the mamba, then the GRU, then all the others. We can clearly see that sequence length is the single most relevant factor for accuracy, and that channel-independent models (Nlinear, Dlinear, PatchTST) score poorly on our task. This is expected, since the different motion parameters are physically correlated, and channel-mixing clearly helps exploiting this correlation.

To analyze the dependence of model performance on the input sequence length (this is a major factor in actual real time deployment) we perform a scan of the best three models (Conformer, Mamba, GRU) over the sequence lengths $T = \{10, 32, 64, 128\}$ and plot the results in Fig. (8)

We can see that the accuracy metrics grow with sequence length. Past $T = 64$ we enter in diminishing-returns territory, and the mean FD actually slightly deteriorates when going from $T = 64$ to $T = 128$.

N. Routing over experts

Inspired by MoE systems in modern LLMs, we decided to try combining different "experts", i.e. model architectures, to give a better overall frame-by-frame prediction. Given the long training runs of the best (heavy) experts, we chose to keep the already trained models and just train a global router. Clearly this doesn't let each architecture become an expert in a particular regime, but it is still an interesting ensemble-like strategy. We chose four experts: the best mamba, conformer and GRU models (at sequence length 128), along with the baseline, and trained a router to predict coefficients to linearly combine the prediction of the four models.

The router is a small MLP that takes as input the per-frame experts' predictions and uncertainties and outputs softmax weights over the four models for the current frame. The router is trained to maximize FD_{gain} with the three experts kept frozen. The router is trained on the validation patients and evaluated on the held-out test patients, both unseen by every expert (more precisely, 85% of the validation patients are used for training, 15%

are for the actual validation (early stopping) and the test patients are for testing). Results are reported in Table (IV).

Model	FD_{gain}
Conformer (single)	0.2159
Mamba (single)	0.2020
GRU (single)	0.1957
Fixed mean	0.2196
Soft router	0.2238
Fixed router-mean weights	0.2221

TABLE IV: Routing performances on the test set

We see that the soft routing strategy found by the model does indeed improve performances. We also collected the average weights produced by the router for each model: 61% for the Conformer, 16% for the Mamba, 15% for the GRU and 8% for the baseline. We immediately see that all models are blended in, and better-performing models are given a larger weight on average. To evaluate the actual role of the per-frame router vs. a fixed linear combination of the models, we evaluated a fixed mean of the 3 best models (33% weight to each model) and a fixed mean of the best models where the weights are given by the previously mentioned averages. We see that active frame-by-frame routing is indeed the best option, although the best fixed linear combination achieves roughly the same performances.

O. Distillation

From the pareto-optimal frontier it is clear that the best performing model is the conformer, followed by the mamba and then by the GRU. Given the large difference in latency between the GRU and the conformer, it'd be ideal to push the GRU to have higher accuracy while still keeping its remarkably low latency. To achieve such goal, we tried knowledge distillation from the conformer onto the GRU.

In particular, distillation happens on two separate spaces: the output space (mean, var) and the latent space, just before the fully connected heads. Both models indeed send the output of their architecture-specific latent space onto a fc mean head and a fc var head. We therefore use the latent space before the fc heads to distill the conformer onto the GRU. Since these latent spaces have different dimensions, we first linearly project the student latent space to the teacher's, and then simply use MSE loss to bring the latent feature closer to each other. For the output space, we use the KL divergence between the normal distributions (mean, var) outputted by the two models. The results are shown in Fig. 10. As we can see, we succedeed in achieving significantly higher FD_{gain} while retaining the very low latency of the GRU model. In future work, this distillation process could be optimized over the various hyperparameters weighing the latent space or output space contributions and the temperature of the teachers' soft labels.

VI. DISCUSSION

VII. CONCLUSIONS

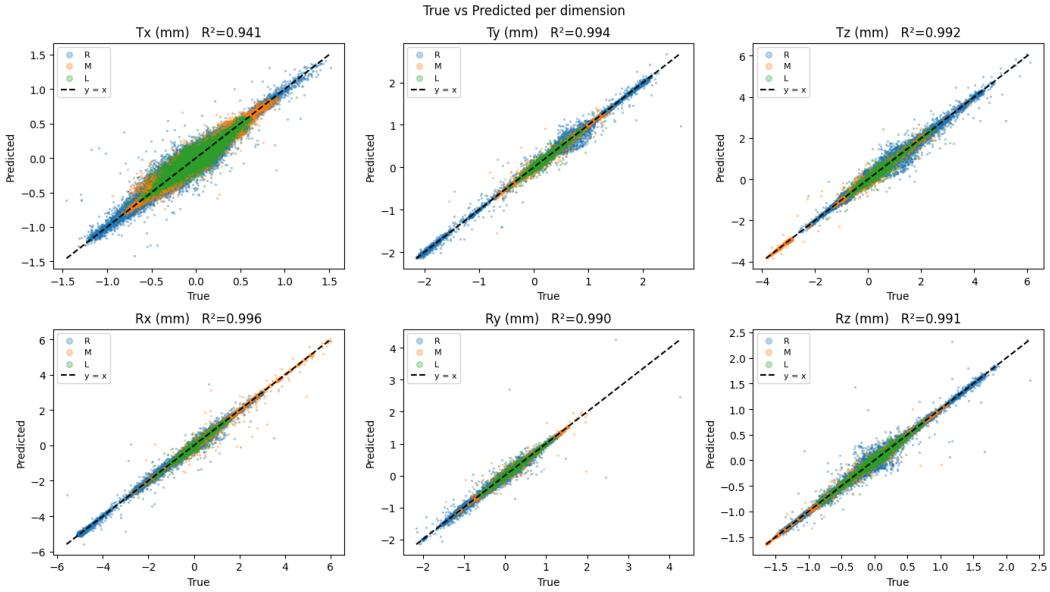
[1] B. Argiento, A. Annovi, S. Capuani, M. Cacioppo, A. Ciardiello, R. Coccurello, S. Giagu, F. Giove, A. Lonardo, F. L. Cicero, A. Maiuro, C. M. Terracciano, M. Merola, M. Montuori, E. Nisticò, P. Perticaroli, B. Rossi, C. Rossi, E. Rossi, F. Simula, and C. Voena, Real-time motion correction in magnetic resonance spectroscopy: Ai solution inspired by fundamental science (2025), arXiv:2509.24676 [physics.med-ph].

[2] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, A time series is worth 64 words: Long-term forecasting with transformers (2023), arXiv:2211.14730 [cs.LG].

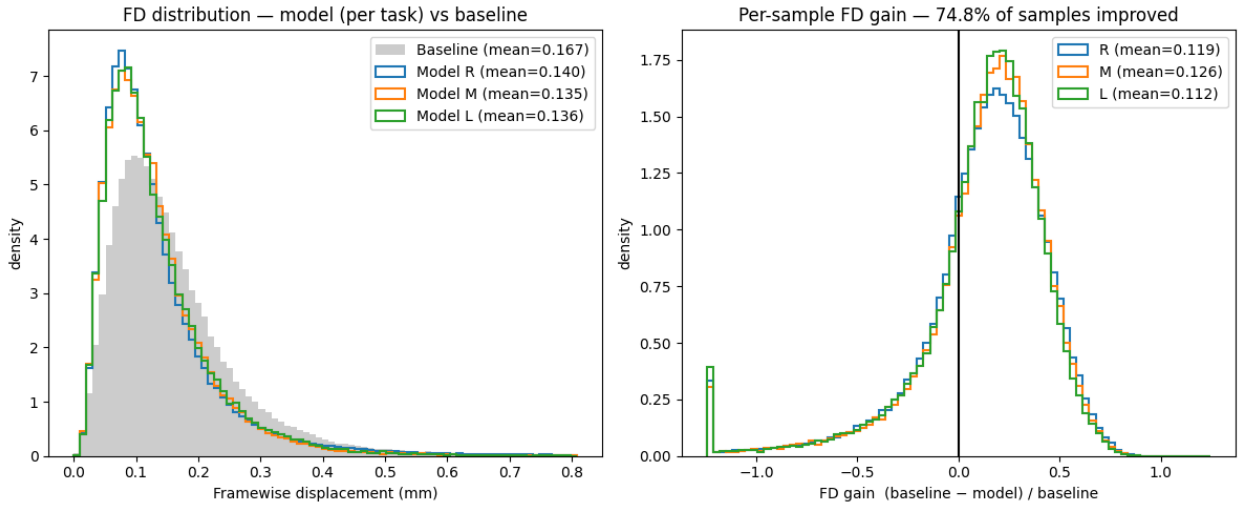
[3] A. Zeng, M. Chen, L. Zhang, and Q. Xu, arXiv preprint arXiv:2205.13504 10.48550/arXiv.2205.13504 (2022).

[4] S.-A. Chen, C.-L. Li, N. Yoder, S. O. Arik, and T. Pfister, Tsmixer: An all-mlp architecture for time series forecasting (2023), arXiv:2303.06053 [cs.LG].

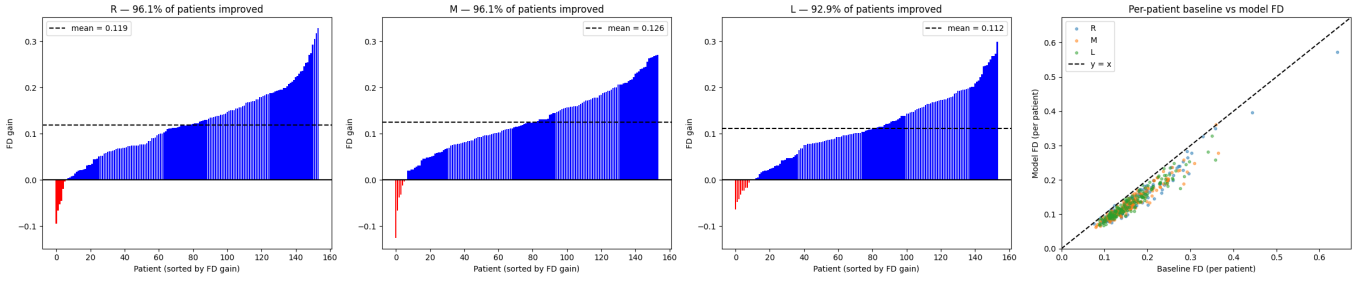
[5] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and R. Pang, Conformer: Convolution-augmented transformer for speech recognition (2020), arXiv:2005.08100 [eess.AS].



(a) True vs. predicted value for each of the six motion dimensions, with per-dimension R^2. All dimensions reach $R^2 > 0.94$.

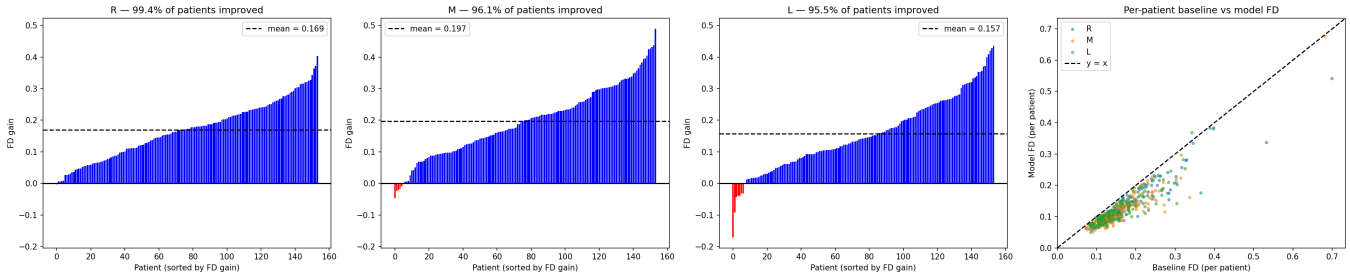


(b) Framewise-displacement distribution of the model vs. the lag-1 baseline (left) and the per-sample FD_{gain} distribution (right): 74.8% of samples are improved over the baseline.

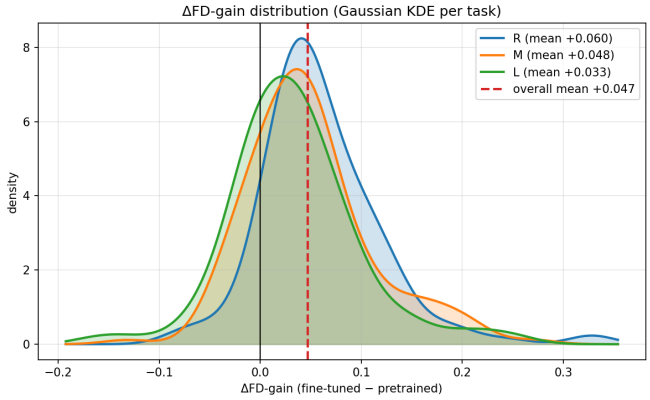


(c) Per-patient mean FD_{gain} sorted within each task (over 92% of patients improved in every task) and per-patient baseline vs. model FD (rightmost panel).

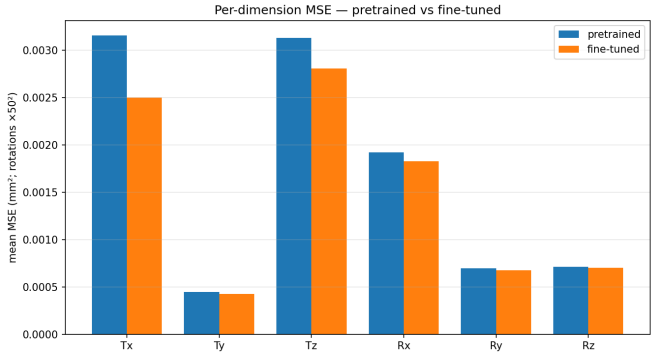
FIG. 2: Reproduction of the GRU results with a single generalist model trained and tested on all three tasks (R+M+L) over disjoint patient splits. (a) per-dimension agreement, (b) framewise-displacement and per-sample gain distributions, (c) per-patient gains.



(a) Per-patient mean FD_{gain} sorted within each task (over 95% of patients improved in every task) and per-patient baseline vs. model FD (rightmost panel).

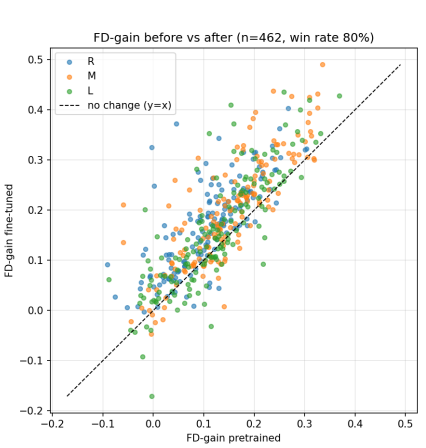


(b) Distribution of the per-patient FD_{gain} improvement (fine-tuned − pretrained); the overall mean shift is +0.047.

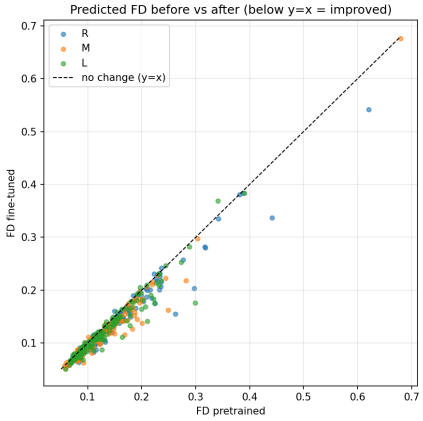


(c) Per-dimension MSE before and after fine-tuning (rotations scaled by 50^2).

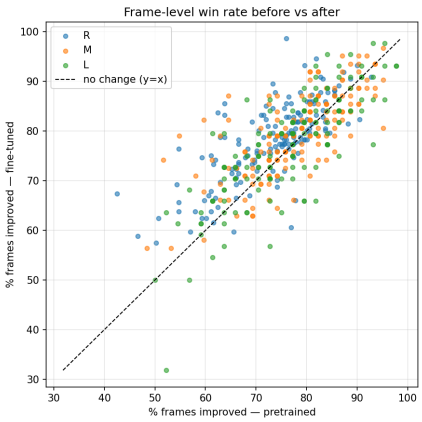
FIG. 3: Per-patient fine-tuning of the generalist GRU model (R+M+L). (a) per-patient gains, (b) distribution of the gain improvement over the pretrained model, (c) per-dimension MSE reduction.



(a) FD_{gain} (win rate 80%).

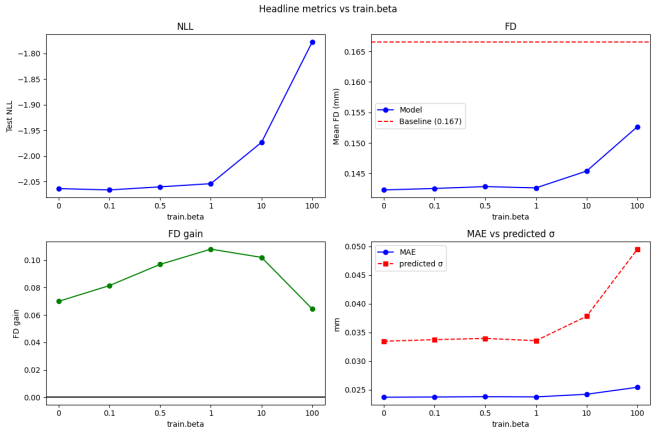


(b) Predicted FD (points below $y = x$ improved).

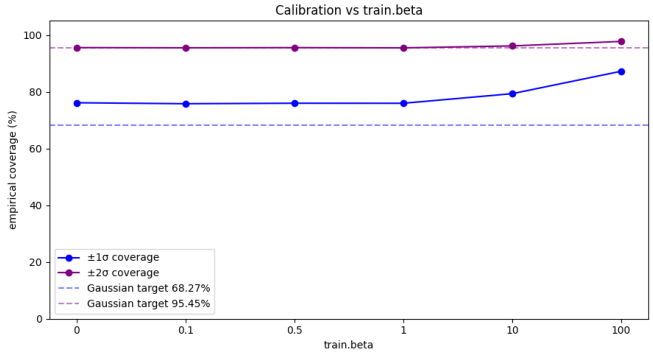


(c) Frame-level win rate.

FIG. 4: Per-patient metrics before vs. after fine-tuning, colored by task. Each point is one patient; the dashed line marks no change ($y = x$).



(a) Headline metrics vs. β: test NLL, mean FD vs. baseline, FD_{gain}, and MAE vs. predicted σ.



(b) Calibration vs. β: empirical coverage of the $\pm 1\sigma$ and $\pm 2\sigma$ predictive intervals against their Gaussian targets.

FIG. 5: β scan of the GRU model. As β grows, FD_{gain} peaks around $\beta = 1$ before the loss over-weights the FD term and uncertainty calibration degrades.

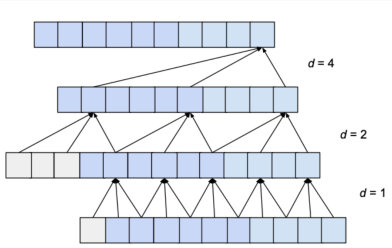


FIG. 6: Schematic diagram of the TCN model

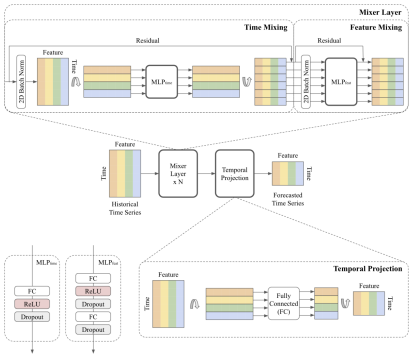


FIG. 7: Schematic diagram of the TSMixer model

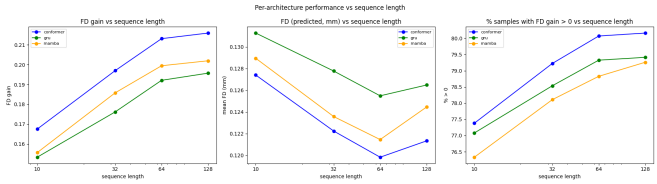


FIG. 8: FD_{gain} as a function of sequence length for the best three models.

TABLE V: Architecture benchmark. Best checkpoint per architecture and sequence length, sorted by mean FD_{gain}. Columns: sequence length, parameter count, checkpoint size, FLOPs, single-frame latency, throughput, mean FD, mean FD_{gain}, percentage of frames improved over baseline, NLL, relative FD degradation at 0.3σ added noise, and noise tolerance.

Arch	Seq	Params	Size (MB)	FLOPs (M)	Lat (ms)	Thrpt (k/s)	Mean FD	FD_{gain}	%>0	NLL	ΔFD@0.3σ %	Tol σ
conformer	128	199,436	0.798	40.38	2.876	42	0.1213	0.2159	80.2	−2.299	+390	0.04
conformer	64	199,436	0.798	20.19	2.800	87	0.1198	0.2131	80.1	−2.290	+396	0.04
gru_distilled	128	174,476	0.698	39.55	0.639	163	0.1236	0.2046	79.3	−2.286	+381	0.04
mamba	128	95,500	0.382	22.73	6.326	12	0.1245	0.2020	79.3	−2.269	+378	0.04
mamba	64	95,500	0.382	11.37	5.046	31	0.1214	0.1995	78.8	−2.272	+389	0.04
conformer	32	199,436	0.798	10.10	2.701	160	0.1222	0.1970	79.2	−2.261	+386	0.04
gru	128	174,476	0.698	39.55	0.643	162	0.1265	0.1957	79.4	−2.244	+371	0.04
gru	64	174,476	0.698	19.79	0.595	279	0.1255	0.1921	79.3	−2.244	+374	0.04
transformer	128	73,228	0.293	0.30	0.963	92	0.1257	0.1907	78.4	−2.254	+375	0.04
mamba	32	95,500	0.382	5.70	4.782	46	0.1236	0.1858	78.1	−2.249	+381	0.04
transformer	64	73,228	0.293	0.16	0.990	199	0.1248	0.1850	78.1	−2.253	+378	0.04
gru	32	174,476	0.698	9.92	0.567	433	0.1278	0.1762	78.5	−2.211	+367	0.03
transformer	32	73,228	0.293	0.08	1.077	351	0.1273	0.1709	77.4	−2.221	+368	0.03
tcn	64	71,276	0.285	10.43	3.194	273	0.1300	0.1700	78.1	−2.202	+360	0.03
tcn	128	71,276	0.285	19.11	3.202	168	0.1317	0.1690	77.5	−2.205	+354	0.03
conformer	10	199,436	0.798	3.16	2.728	369	0.1274	0.1675	77.4	−2.206	+370	0.03
tcn	32	71,276	0.285	6.08	3.332	316	0.1315	0.1569	77.1	−2.176	+355	0.03
mamba	10	95,500	0.382	1.79	4.049	131	0.1289	0.1556	76.3	−2.191	+363	0.03
gru	10	174,476	0.698	3.12	0.558	696	0.1313	0.1533	77.1	−2.166	+355	0.03
transformer	10	73,228	0.293	0.03	0.950	803	0.1311	0.1478	76.1	−2.183	+356	0.03
tcn	10	71,276	0.285	3.10	3.253	315	0.1346	0.1392	76.3	−2.150	+344	0.03
TSMixer	64	18,042	0.072	0.74	2.009	480	0.1383	0.1160	73.9	−2.168	+314	0.03
TSMixer	128	67,578	0.270	2.66	1.966	474	0.1478	0.0857	71.5	−2.071	+278	0.02
TSMixer	32	5,562	0.022	0.22	1.822	529	0.1518	0.0609	71.2	−2.011	+278	0.02
dlinear	32	1,782	0.007	0.00	0.916	1109	0.1534	0.0539	71.2	−0.865	+233	0.02
nlinear	32	396	0.002	0.00	0.450	2229	0.1533	0.0538	70.8	−1.028	+234	0.02
nlinear	64	780	0.003	0.00	0.451	1991	0.1540	0.0534	70.9	−1.012	+231	0.02
dlinear	64	3,510	0.014	0.01	0.946	1075	0.1538	0.0500	69.8	−1.496	+229	0.02
nlinear	128	1,548	0.006	0.00	0.453	2222	0.1557	0.0495	69.6	−0.479	+229	0.02
dlinear	128	6,966	0.028	0.01	0.930	1111	0.1555	0.0486	69.2	−1.245	+228	0.02
dlinear	10	594	0.002	0.00	0.982	1059	0.1521	0.0469	69.5	−1.749	+224	0.02
nlinear	10	132	0.001	0.00	0.447	2276	0.1525	0.0413	68.5	−1.676	+222	0.02
patchTST	128	23,330	0.093	19.17	1.482	21	0.1553	0.0397	68.4	−2.059	+193	0.02
patchTST	32	18,722	0.075	4.79	1.491	67	0.1551	0.0355	69.5	−1.986	+207	0.02
patchTST	64	20,258	0.081	9.58	1.511	42	0.1571	0.0347	69.0	−1.895	+216	0.01
patchTST	10	17,666	0.071	1.50	1.500	134	0.1547	0.0344	69.4	−1.982	+203	0.02
TSMixer	10	1,734	0.007	0.04	1.938	517	0.1664	−0.0002	53.1	−0.143	+263	0.00

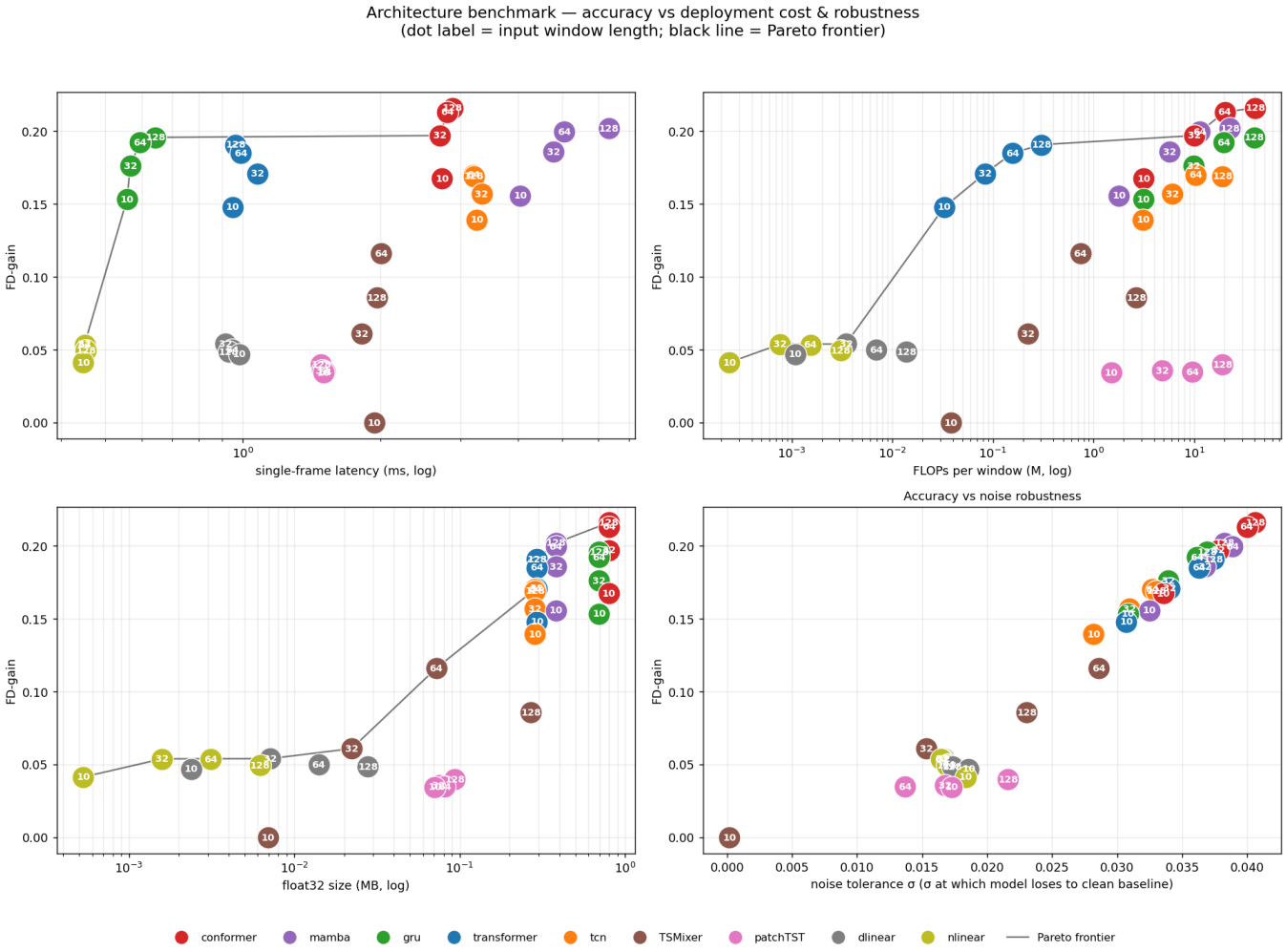


FIG. 9: Pareto-optimal frontier visualizations. FD_{gain} (accuracy) is plotted against the different cost axes (latency, FLOPs, size) and against noise tolerance. The conformer leads on accuracy, the mamba follows, and the GRU offers the best accuracy-per-latency trade-off.

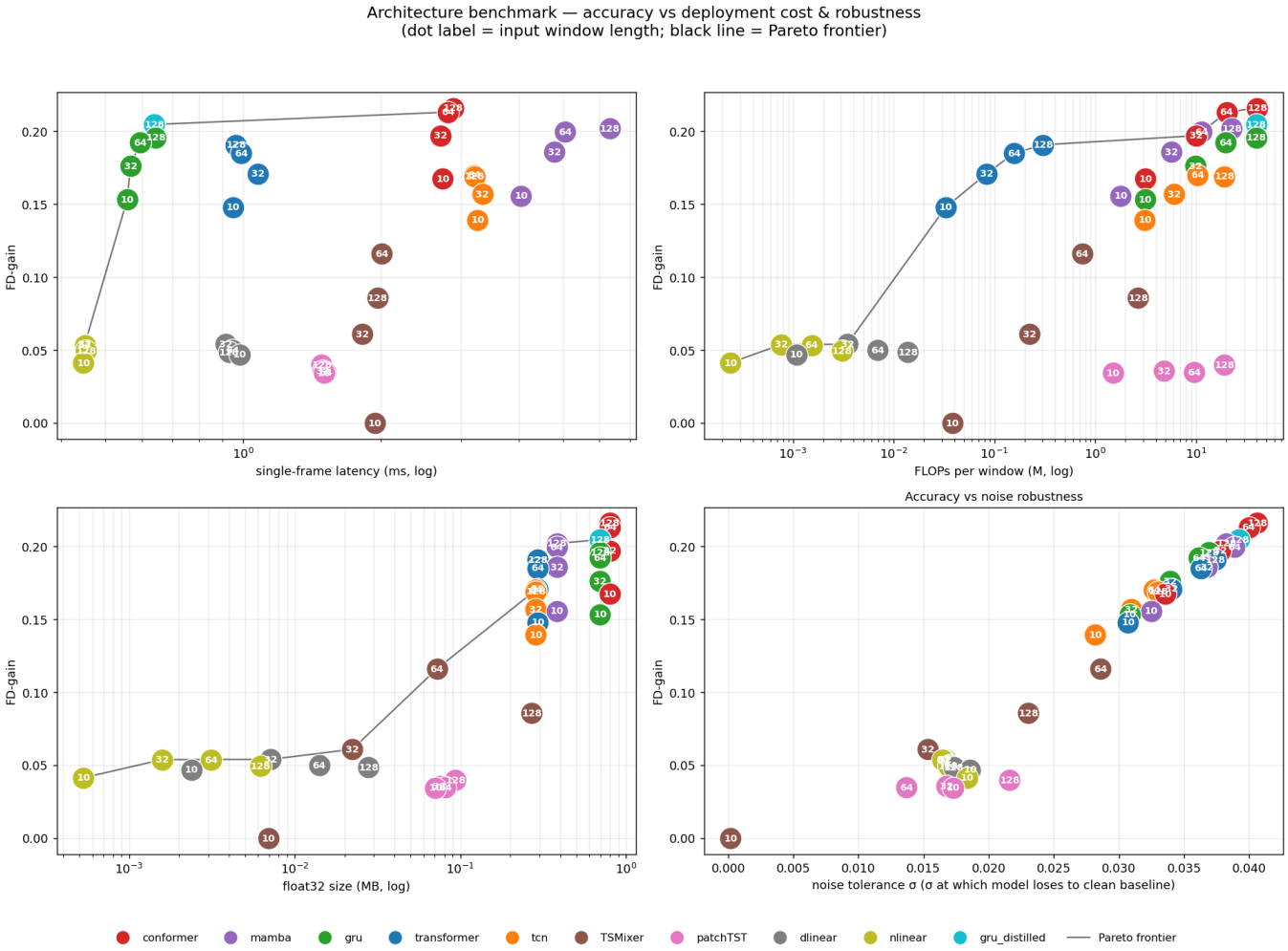


FIG. 10: Pareto-optimal frontier visualizations including the conformer-distilled GRU. The distilled GRU attains a substantially higher FD_{gain} than the vanilla GRU while retaining the GRU's very low latency, moving it toward the accuracy of the heavier conformer/mamba experts.